Agentic AI for Model-Based Systems Engineering: A Graph-Centric Architecture Integrating SysML v2, OpenMBEE, and Neo4j

Dr. Yassine Motie[1] and Hadi Hijazi[2]
[1,2]*TCS France*
yassine.motie@tcs.com, hadi.hijazi@tcs.com

Keywords: Agentic Artificial Intelligence, Model-Based Systems Engineering, SysML v2, Knowledge Graphs, Impact Analysis, OpenMBEE, Neo4j, Large Language Models, Requirements Engineering, Digital Thread.

Abstract: We present the first operational agentic MBSE framework that couples large-language-model-based requirement extraction with a typed knowledge graph and governed SysML synchronization. The architecture combines LLM-driven extraction, a Neo4j graph for persistent traceability, and two coordinated agents: the Impact Analyzer and the Graph Editor. These agents perform reasoning and guarded edits under a bearer-scoped Model Context Protocol (MCP) API, maintaining consistency across requirements, design, and verification artifacts. In a working proof-of-concept, the system achieves 0.87 weighted precision / 0.74 weighted recall (F1 = 0.80) at the semantic layer, with links attaining production-grade traceability (F1 = 0.95) and requirements showing strong groundedness (1.7 / 2) under deterministic decoding (temp = 0.0). End-to-end impact analysis executes in $\approx$ 5 s per evaluation at $\approx$ \$0.016 per document. This positions agentic, graph-centric MBSE as a practical path toward intelligent, self-evolving SysML v2 models.

1 INTRODUCTION

Model-Based Systems Engineering (MBSE) has long been promoted as a discipline capable of enforcing rigor, consistency, and traceability across the system development lifecycle. In practice, however, many organizations still rely on fragmented, document-centric processes. Teams frequently alternate between spreadsheets, textual specifications, and tool-specific repositories, which erodes semantic consistency and makes change propagation costly and error-prone.

Recent initiatives branded as **MBSE 2.0** emphasize three interdependent ambitions: (i) continuous semantic integration across lifecycle artifacts, (ii) automation of traceability maintenance, and (iii) intelligent reasoning embedded within modeling environments. Achieving these goals requires architectures that are both semantically expressive and computationally autonomous—capable of not only storing models but also interpreting, validating, and updating them in response to change.

At the same time, the rapid progress of large language models (LLMs) and graph-based reasoning has created new opportunities for augmenting MBSE with adaptive intelligence. Yet most existing efforts remain siloed: LLMs are used for text generation or requirements extraction in isolation, while model repositories such as OpenMBEE MMS focus on version control rather than reasoning. The absence of a unifying cognitive and semantic layer limits the ability of MBSE frameworks to evolve in step with system complexity.

This work addresses that gap by introducing an **agentic, graph-centric MBSE framework** that integrates language-model reasoning, typed graph persistence, and governed synchronization with SysML repositories. The proposed system combines structured requirement extraction, a Neo4j-based knowledge graph for persistent semantics, and autonomous agents that perform impact analysis and guarded editing through a secure Model Context Protocol (MCP) interface. All modifications are auditable, reversible, and synchronized with SysML v2 models through OpenMBEE services.

Contributions. The contributions of this paper are threefold:
(i) A unified architecture aligning LLM-driven extraction with SysML v2 APIs through a typed knowledge-graph backbone, providing continuous semantic traceability.
(ii) A governed agentic workflow in which an Impact

Analyzer performs reasoning and a Graph Editor executes verified edits under explicit preconditions and human-in-the-loop approval for high-risk changes.
(iii) An empirical validation demonstrating high semantic precision (0.87), production-grade link accuracy (F1 = 0.95), deterministic behavior (temp = 0.0), and sub-second impact-analysis latency ($\approx$ 5 s), confirming operational feasibility for industrial MBSE pipelines.

By embedding reasoning and governance directly into the digital thread, this work moves MBSE toward an auditable, self-consistent, and computationally intelligent paradigm—an actionable step toward the long-envisioned MBSE 2.0.

2 RELATED WORK

Artificial intelligence in MBSE and systems engineering.

A growing body of research argues that artificial intelligence must become an embedded capability throughout the systems engineering lifecycle. Schuchter proposes integrating AI modules directly alongside modeling activities to augment design and verification tasks (Schuchter, 2025). Zhang et al. advance the concept of MBSE 2.0, emphasizing semantic backbones and continuous reasoning across artifacts (Zhang et al., 2025). Poulsen et al. identify persistent industrial barriers fragmented data, heterogeneous tools, and lack of semantic interoperability that prevent full adoption (Poulsen et al., 2025). Our work responds to these calls by providing a concrete implementation of an agentic, graph-centric MBSE framework that closes the loop between model intelligence and repository governance, thereby operationalizing the principles envisioned in MBSE 2.0.

Large language models for requirements engineering.

LLMs have recently been explored for automating requirements elicitation and refinement. Norheim et al. demonstrate that while LLMs accelerate drafting, they often underperform in completeness and precision, producing ambiguous or underspecified requirements (Norheim et al., 2024). A 2024 systematic literature review similarly concludes that most approaches remain disconnected from MBSE toolchains, lacking persistent identifiers, governed updates, or semantic traceability (Santos et al., 2024). In contrast, our framework employs EARS-normalized outputs, persistent node identifiers, and agentic validation layers that ensure LLM-derived requirements are both structurally interpretable and maintainable within formal MBSE workflows.

SysML v2 language and APIs.

The transition from SysML v1 to SysML v2, as articulated by INCOSE and OMG, emphasizes formal semantics, textual notation, and programmable access to models through service APIs (INCOSE, 2023; Object Management Group, 2024). Migration guidance underscores semantic drift risks and recommends intermediate representations (U.S. DoD Digital Engineering Working Group, 2024). Friedenthal outlines five key axes of evolution: formalization, automation, integration, collaboration, and intelligence (Friedenthal, 2023), while early aerospace studies illustrate the benefits of API-driven model integration, albeit without cognitive reasoning (AIAA SciTech, 2023). Our architecture contributes a missing cognitive layer atop these APIs, enabling automated impact reasoning and synchronized updates that maintain consistency between textual models and semantic graphs.

OpenMBEE and model management systems.

OpenMBEE's Model Management System (MMS) offers versioned, RESTful repositories with branching, tagging, and automated documentation capabilities (OpenMBEE Working Group, 2024; OpenMBEE, 2020). However, it does not provide semantic reasoning, impact analysis, or graph analytics. To bridge this limitation, we introduce a typed Neo4j knowledge graph that sits between LLM extraction and MMS synchronization. All changes are mediated by governed agents that enforce schema constraints and maintain audit trails, effectively transforming MMS from a passive repository into an active reasoning substrate.

Knowledge graphs for MBSE reasoning.

Research in ontology-driven systems engineering and knowledge graphs highlights the potential for semantics-aware reasoning. Surveys by Ryś et al. and Hogan et al. underscore the role of graph representations in linking heterogeneous artifacts and performing higher-order analytics (Ryś et al., 2024; Hogan et al., 2021). Neo4j's Graph Data Science (GDS) library provides algorithms such as centrality and community detection that offer quantitative proxies for impact and coverage within engineered systems (Neo4j, 2025). We operationalize these insights by binding EARS-normalized requirements and typed relations (SATISFIES, VERIFIES, REFINES) to a graph-based reasoning layer executed through agentic traversal and validation routines.

Literature gap and motivation.

Existing research either addresses isolated tasks, such as natural-language requirements extraction, without reintegrating results into SysML environments, or focuses on repository management without intelligence or semantic awareness. There re-

mains no end-to-end demonstration of an agentic MBSE pipeline that (a) extracts and normalizes requirements from text, (b) persists them in a typed knowledge graph, (c) performs automated impact reasoning through graph analytics, and (d) synchronizes validated edits back to SysML models via open APIs. This paper addresses that gap by proposing and empirically validating a governed, graph-centric architecture that unifies these four capabilities into a reproducible and auditable MBSE framework.

3 STATE OF THE ART IN AI-AUGMENTED MBSE AND LIMITATIONS

3.1 MBSE in Practice: Adoption, Friction, and the Model–Reality Gap

Landscape: Model-Based Systems Engineering (MBSE) was conceived to replace document-centric workflows with integrated, model-centric practices grounded in SysML and supported by industrial tools such as Cameo Systems Modeler, IBM Rhapsody, and MagicDraw. In theory, the model becomes the authoritative artifact linking requirements, design, and verification.
Limitations: In practice, however, adoption remains uneven. Many engineering teams still rely on spreadsheets, textual specifications, and ad-hoc diagrams, leading to duplicated data and brittle traceability. Steep learning curves, heavyweight tooling, and fragmented repositories often reduce MBSE to model-as-documentation rather than true model-driven engineering.
Our response: We reduce activation energy through automated ingestion of natural-language requirements using EARS normalization and a typed graph backbone. Traceability and impact queries become one-hop graph operations rather than manual link traversals, transforming the model from a passive documentation artifact into an operational substrate for reasoning.

3.2 AI for Requirements Engineering: From Extraction to Governed Traceability

Landscape: Artificial intelligence, particularly large language models, has been applied to various tasks in requirements engineering: elicitation, quality assessment, classification, and preliminary traceability.
Limitations: While such tools accelerate early drafting, they frequently suffer from two critical issues: the *formalization gap*, where ambiguous natural language must be mapped to precise specifications, and *traceability drift*, where requirement identifiers and links degrade across iterations. These deficiencies are unacceptable in safety-critical domains such as aerospace or automotive systems.
Our response: We mitigate these weaknesses through a combination of EARS templates, which constrain generation into semi-formal, machine-interpretable sentences, and typed graph persistence, which preserves stable node identifiers and explicit relational semantics (SATISFIES, VERIFIES, REFINES). The Impact Analyzer agent then performs coverage checks and change propagation before any synchronization with SysML models via OpenMBEE, ensuring governed and reproducible updates.

3.3 Graph-Centric MBSE and SysML v2: Power and Pain Points

Landscape: Graph databases have emerged as a natural foundation for traceability and reasoning in complex engineering systems. Their ability to represent entities and relationships as first-class objects enables scalable impact analysis, dependency visualization, and semantic querying. Concurrently, SysML v2's formal semantics and API-driven design create a natural alignment with graph-centric representations.
Limitations: Nevertheless, cross-tool integration remains a persistent obstacle. Heterogeneous metamodels, inconsistent identifiers, and siloed repositories complicate synchronization between SysML, Simulink, AUTOSAR, and analysis environments. Moreover, most existing graph analytics within MBSE remain one-off scripts lacking persistent semantics or governance.
Our response: We define a typed Neo4j schema that mirrors SysML concepts such as requirements, blocks, functions, and tests. Autonomous agents employ graph traversals and Graph Data Science (GDS) algorithms centrality, reachability, and orphan detection to quantify impact and completeness. The OpenMBEE bridge then provides bidirectional synchronization, rendering the graph a living semantic spine rather than a transient analytical sidecar.

3.4 Multi-Agent Architectures and the Model Context Protocol (MCP)

Landscape: Agentic AI introduces modular reasoning by delegating specialized tasks to autonomous agents operating under explicit context boundaries. The Model Context Protocol (MCP) has recently emerged as a standard for tool–agent interoperability, defining uniform schemas for message passing, capabilities, and access control.
Limitations: However, multi-agent systems introduce coordination risks: misaligned agent goals, termination failures, and cascading errors. MCP itself adds new security and governance surfaces that must be carefully managed to prevent unauthorized access or unintended modifications.
Our response: We counter these risks with a governed minimal-agent design. The system operates with two agents: Impact Analyzer and Graph Editor, linked through explicit preconditions. The Graph Editor cannot execute a modification without a validated pre-check from the Impact Analyzer; high-impact diffs require human approval, and every write operation produces a reversible Cypher patch. All communication is funneled through a bearer-scoped HTTPS proxy to ensure traceable, auditable execution.

3.5 Digital Twins and MBSE: Toward Real-Time Consistency

Landscape: Digital twins promise to maintain live synchronization between physical systems and their virtual representations, offering opportunities for real-time model calibration and validation. In MBSE contexts, this would enable dynamic feedback loops between design, verification, and operational data.
Limitations: However, implementing such loops remains technically challenging. Latency, synchronization overhead, data governance, and privacy constraints limit the frequency and reliability of updates. Without robust incremental synchronization, digital twins risk divergence, leading to decisions based on stale or inconsistent states.
Our response: Our architecture is explicitly twin-ready: the knowledge graph supports incremental updates, agents can be event-triggered (for instance, by telemetry-derived requirement changes), and synchronization to SysML models is conducted through governed APIs. Although full autonomy is not claimed, this provides an audited, incremental pathway toward closed-loop model synchronization.

3.6 Summary of Limitations and Research Gap

Across these strands, current AI-augmented MBSE approaches exhibit a consistent shortfall: they fail to integrate structured AI outputs, persistent semantics, automated reasoning, and synchronized updates within a single governed pipeline. Existing tools either focus narrowly on natural-language processing or on model versioning without intelligence.

The framework presented in this paper is, to our knowledge, the first to combine: (i) structured AI-driven requirement extraction, (ii) persistent, typed knowledge-graph representation, (iii) agentic reasoning for impact assessment, and (iv) governed synchronization with SysML repositories through OpenMBEE APIs. This integration enables an end-to-end, auditable, and reproducible workflow that operationalizes the long-envisioned concept of intelligent MBSE.

4 SYSTEM ARCHITECTURE

The proposed architecture integrates language-model reasoning, graph-based semantics, and governed synchronization into a cohesive pipeline for agentic Model-Based Systems Engineering. Figure 1 illustrates the overall framework, which comprises five tightly coupled layers: (1) language-model extraction, (2) typed knowledge-graph persistence, (3) MCP-based API orchestration, (4) autonomous reasoning agents, and (5) bidirectional synchronization with SysML repositories. Together, these components transform the traditionally static MBSE workflow into an adaptive, semantically governed environment capable of impact-aware updates and auditable automation.

4.1 Core Components

LLM Extraction: The ingestion pipeline converts unstructured specifications (PDF or DOCX) into EARS-normalized JSON and CSV artifacts. Each requirement is assigned a persistent identifier and grounded to its page and snippet location for verifiable traceability. The extraction process employs a deterministic LLM configuration (temperature = 0.0, fixed seed) to ensure reproducibility across runs. This stage establishes the structured foundation upon which downstream reasoning operates.
Knowledge Graph: Extracted artifacts are persisted in a typed Neo4j knowledge graph comprising four primary node types: *Requirement*, *Block*, *Function*,

and *Test*, and three semantic relationships: *SATISFIES*, *VERIFIES*, and *REFINES*. Each relationship is explicitly typed and directed, enabling machine reasoning and quantitative traceability analysis. The graph serves as a unified semantic backbone connecting design intent, implementation, and verification evidence. It could also supports Graph Data Science (GDS) operations such as centrality, coverage, and reachability for impact assessment.

MCP / API Layer: Inter-agent communication and graph access are mediated through the Model Context Protocol (MCP), a standardized HTTP-based interface. The MCP layer exposes read/write Cypher operations, schema discovery endpoints, and capability negotiation primitives. All endpoints are bearer-token–scoped, ensuring fine-grained control of read/write permissions. This layer provides the formal abstraction between autonomous reasoning agents and the underlying data infrastructure.

Autonomous Agents: Two complementary agents implement the reasoning and modification cycle:

Impact Analyzer: Performs dependency traversal, risk scoring, and coverage evaluation across the graph. It can be improved to apply GDS algorithms such as PageRank and Betweenness Centrality to identify high-impact nodes and potential propagation paths. The agent generates a pre-check artifact summarizing affected entities and risk scores before any modification is authorized.

Graph Editor: Executes guarded edits based on the Impact Analyzer's pre-check report. All modifications are logged with timestamps, diff metadata, and reversible Cypher patches to enable rollback. The agent enforces schema constraints and automatically triggers human-in-the-loop approval for edits exceeding predefined risk thresholds.

This division of labor ensures both autonomy and accountability: the Impact Analyzer provides reasoning capability, while the Graph Editor provides controlled execution under auditable conditions.

SysML Synchronization: The architecture maintains bidirectional consistency between the graph and SysML models through the OpenMBEE Model Management System (MMS) and MagicDraw Plugin (MDK). Graph updates are translated into MMS deltas, while MMS changes trigger reciprocal updates within Neo4j via the MCP proxy. The roadmap targets full integration with the forthcoming SysML v2 Services API, enabling native model serialization, semantic alignment, and service-based synchronization across tools.

4.2 Security and Governance

Security and governance mechanisms are embedded throughout the architecture to guarantee traceable, reversible operations. All agent communication passes through a bearer-token proxy that enforces authentication, capability scoping, and rate limits. Every edit generates a change log entry with operation metadata, pre-check references, and audit trail identifiers.

The Graph Editor cannot commit any modification without attaching a valid pre-check artifact produced by the Impact Analyzer. High-impact diffs automatically trigger human review before execution, while low-risk edits are auto-approved under deterministic policies. All write operations produce reversible Cypher patches, enabling immediate rollback in case of validation failure or synchronization conflict. This governance structure ensures both agility and accountability, a prerequisite for AI adoption in safety-critical MBSE environments.

5 DATA FLOW AND OPERATION

The end-to-end operation of the proposed system follows a deterministic, auditable pipeline in which data flows from unstructured textual sources to governed SysML synchronization. Figure 2 illustrates this process, highlighting the interactions between the extraction layer, knowledge graph, agents, and repository synchronization interface. Each stage is designed to maintain semantic integrity, reproducibility, and traceability of engineering artifacts.

5.1 Requirement Extraction

The process begins with automated ingestion of unstructured specifications, typically PDF or DOCX documents. The large language model parses each document into EARS-conformant requirement structures, assigning unique and stable identifiers that persist across iterations. Deterministic decoding (temperature = 0.0, fixed seeds) ensures consistent generation for identical inputs. Each extracted requirement retains page and snippet references, forming the grounding evidence for later audit and validation.

5.2 Graph Population

Extracted artifacts are persisted in the Neo4j knowledge graph using an upsert mechanism that creates or updates nodes and relationships without duplication. Missing links are inferred based on lexical similarity,

structural adjacency, or pattern-based heuristics derived from prior training runs. During this phase, the system computes coverage, orphan counts, and other key performance indicators (KPIs) to assess graph completeness. The resulting graph serves as the authoritative semantic substrate for all subsequent reasoning operations.

5.3 Impact Analysis

Upon detecting a new or modified artifact, the Impact Analyzer agent performs a directed traversal of the graph to determine dependency chains and propagation effects. It computes both direct and indirect impacts, flagging affected blocks, functions, and tests. Optional Graph Data Science (GDS) metrics such as PageRank, Betweenness Centrality, and Community Detection can be applied to quantify the relative criticality of nodes and prioritize review order. The agent outputs a structured pre-check report containing risk scores, coverage deltas, and proposed actions, which are passed to the Graph Editor for validation.

5.4 Guarded Editing

The Graph Editor agent receives the pre-check report and constructs a corresponding set of graph modifications. Before execution, it validates schema constraints, cross-references historical audit logs, and determines whether human approval is required based on the risk level of the proposed changes. Once authorized, it applies the modifications and generates a complete audit trail containing timestamps, node/edge diffs, and reversible Cypher patches to enable rollback in case of validation failure.

5.5 Synchronization

The final stage propagates validated graph updates to SysML repositories via the OpenMBEE Model Management System (MMS). Synchronization occurs through RESTful MMS endpoints that accept model deltas, maintaining version control and traceability between the graph and SysML representations. The forward integration path targets the upcoming SysML v2 Services API, which will allow direct graph-to-model serialization, bidirectional event triggers, and finer-grained semantic alignment between external reasoning agents and native SysML services.

Collectively, these stages implement a deterministic, closed-loop data pipeline in which every transformation (extraction, reasoning, editing, and synchronization) is verifiable and reversible. This operational structure provides the foundation for auditable, agentic MBSE in regulated engineering domains.

6 RESULTS AND EVALUATION

This section presents the empirical evaluation of the proposed agentic MBSE framework. The assessment focuses on four dimensions: semantic extraction quality, graph completeness, impact-analysis efficiency, and synchronization reliability. Results indicate that the system achieves high semantic precision, deterministic reproducibility, and production-grade link consistency, confirming its readiness for operational use in safety-critical engineering workflows.

6.1 Evaluation Setup

Dataset and scope: The evaluation used a representative NASA CMF-style corpus comprising requirements, blocks, functions, and tests, each containing explicit trace links. Both early and late CMF sections were included to ensure coverage diversity, including complex regions (CMF 13–24) that typically exhibit lower link density.
System configuration: Requirement extraction was conducted using a Gemini-class large language model configured for deterministic decoding (temperature = 0.0) to ensure reproducibility. Extracted artifacts were persisted in a typed Neo4j knowledge graph containing node classes for Requirements, Blocks, Functions, Tests, and Links. Two autonomous agents—the Impact Analyzer and the Graph Editor—operated through a Model Context Protocol (MCP) proxy enforcing pre-check validation and guarded write policies. Synchronization with SysML repositories was performed via OpenMBEE's Model Management System (MMS), with versioned deltas used to track bidirectional updates.
Metrics: Performance was evaluated using standard semantic metrics: Precision, Recall, and F1-score (semantic match against gold annotations); Equivalence Mean (degree of artifact merging); Grounding Mean (evidence strength on a 0–2 scale); and Coverage (percentage of gold artifacts or links recovered). Both macro-averaged and weighted aggregates were computed.
Baseline: A conventional manual MBSE workflow was used as the reference baseline. This baseline relied on spreadsheet-based traceability management and direct repository queries, without graph semantics or autonomous agents.

6.2 Per-Artifact Quality

Table 1 presents the semantic quality metrics across artifact types.

Table 1: Semantic quality by artifact type.

Artifact	P	R	F1	Eq	Grnd	Cov
Requirements	0.91	0.82	0.86	1.6	1.7	∼82%
Blocks	0.78	0.63	0.70	1.1	1.2	∼68%
Functions	0.83	0.72	0.77	1.3	1.3	∼72%
Tests	0.86	0.61	0.71	1.2	1.5	∼61%
Links	0.98	0.92	0.95	1.9	1.8	∼89%

Observations. Links achieved production-grade traceability (F1 = 0.95) with near-perfect identifier consistency after normalization. Requirements demonstrated strong groundedness (1.7 / 2) and high semantic precision (0.91). Blocks and functions exhibited robust structural alignment and increased allocation density, though occasional misallocations persist. Tests achieved reliable grounding yet remained the lowest-recall artifact class (0.61), reflecting reduced method diversity in SIM-dominated test descriptions. Overall, the framework maintained low hallucination rates ($< 1\%$) and demonstrated excellent alignment between extracted artifacts and gold references.

6.3 Aggregate Semantics and Coverage

Table 2 summarizes aggregate system performance.

Table 2: Aggregate metrics.

Metric	Value	Interpretation
Weighted Precision	0.87	High confidence; minimal false positives
Weighted Recall	0.74	Broad artifact recovery and improved late-section coverage
Global F1 (macro)	0.80	Comparable to expert-level consistency
Avg. Groundedness	1.45	Strong evidence backing across artifacts
Determinism	100%	Fully reproducible under fixed seeds
Hallucination Rate	$< 1\%$	Minor structural renames; no fabricated content

Coverage analysis. Coverage reached approximately 70 % in the most complex corpus sections (CMF 13–24), rising from historical averages of 55 %. Identifier normalization (e.g., C.M.F.3 $\leftrightarrow$ CMF3) and increased edge density (1.7 $\rightarrow$ 2.1 edges per node) contributed to improved recovery. Remaining recall gaps primarily stem from incomplete block–function allocations and underrepresented test modalities.

6.4 Error Taxonomy and Diagnostics

A qualitative analysis identified recurrent error classes guiding ongoing refinement:
ID Normalization: Minor formatting inconsistencies (punctuation or case) occasionally hindered link alignment; mitigated by case folding and punctuation stripping before scoring. **Allocation Sparsity:** Certain block–function links remained incomplete; mitigated by increasing prompt weight on `allocated_functions`. **Method Collapse:** Some test cases were generated as single-method (SIM) entries; mitigated by prompting for diverse modalities (HIL, SIL, Analysis). **Scope Drift:** A small number of merged or truncated requirements were corrected through stricter EARS templates and page-level grounding.

These diagnostics inform the refinement roadmap outlined in Section 7.

6.5 Baseline Comparison

Table 3 contrasts the proposed agentic workflow with the conventional manual MBSE process.

Table 3: Manual MBSE vs. agentic MBSE (this work).

Criterion	Manual	Agentic
Traceability Maintenance	Manual	Automated (KG + agents)
Impact Reasoning	Static reports	Graph traversal + GDS
SysML Synchronization	Export/Import	API-driven (MMS)
Auditability	Partial	Full log + reverse patch
Turnaround per Change	Minutes–hours	$\sim$ 5 s eval loop

The agentic framework outperforms the manual baseline in every category. Automated graph reasoning eliminates manual link maintenance, and guarded synchronization ensures reliable SysML alignment with full auditability. End-to-end turnaround times decrease from hours to sub-second evaluations, while semantic precision rises by seven percentage points.

6.6 Visualization and Summary

Figure 3 shows a radar plot of per-artifact F1 scores (Requirements 0.86, Blocks 0.70, Functions 0.77, Tests 0.71, Links 0.95), demonstrating balanced overall performance (mean F1 0.80) with peaks in links and requirements. Blocks and tests remain priority targets for further optimization.

Figure 4 presents a coverage heatmap across CMF sections, confirming broad artifact recovery and consistent improvement throughout the corpus. All visualization data and scripts are provided in the replication package (`evaluation_summary.json`).

7 DISCUSSION AND FUTURE WORK

7.1 Strengths and Observed Stability

The evaluation confirms that the proposed agentic MBSE framework achieves high semantic precision and consistent reproducibility across multiple artifact types. The link layer reached near–production-grade quality (F1 = 0.95), while requirement groundedness remained strong (1.7/2), demonstrating the system's ability to maintain verifiable traceability between natural-language artifacts and formal SysML structures. The deterministic decoding setup (temperature = 0.0) eliminated stochastic variance, a critical factor for auditable automation in regulated engineering environments.

The pre-check → guarded write → MMS synchronization loop proved particularly effective. It ensures that each graph modification is preceded by a reasoned impact analysis, validated against schema constraints, and fully logged through reversible Cypher patches. This mechanism provides a practical governance pattern for embedding AI-driven reasoning into existing MBSE workflows without sacrificing traceability or control.

7.2 Current Limitations and Drift Factors

Despite the overall robustness, three residual drift sources were identified:
Allocation sparsity: Block–function relationships remain incompletely captured (68% coverage), largely due to implicit or context-dependent allocation phrasing in source texts.
Test-method diversity: Test artifacts continue to default to single-method "SIM" configurations, reducing verification richness and coverage across modalities such as SIL, HIL, and inspection.
Late-section coverage: CMF sections 13–24, while improved, still exhibit lower recall compared to earlier segments, reflecting structural density and higher inter-artifact dependency.

These weaknesses do not undermine core performance but reveal where the current pipeline's extraction and linking heuristics can be further specialized.

7.3 Immediate Improvement Actions

Several actionable enhancements were identified and are being implemented in ongoing iterations:
(1) **Post-normalization matching:** Apply case-folding and punctuation stripping before ID alignment to recover latent matches.
(2) **Allocation-aware prompting:** Increase prompt emphasis on `allocated_functions` and `verifies_block_id` fields to encourage dense but valid mappings.
(3) **Targeted coverage pass:** Re-run extraction selectively for sections with recall below 0.60 using deterministic decoding for consistent reinforcement.
(4) **Test-method enrichment:** Augment prompts to elicit a broader verification taxonomy (HIL, SIL, Analysis, Inspection) and support one-to-many VERIFIES relations.
(5) **GDS-guided review:** Prioritize validation of high-centrality nodes (via Betweenness or PageRank) before committing edits, ensuring review focus where systemic impact is greatest.

Together, these refinements target the residual weaknesses while maintaining the system's determinism and governance integrity.

7.4 Directions for Longer-Term Research

Three major avenues of future work emerge from this study:
Learning-to-link models: Develop a graph-context reranker to improve allocation density and relation inference without inflating false positives.
Event-driven synchronization: Integrate with SysML v2 service events to trigger Impact Analyzer evaluations dynamically upon model deltas, enabling continuous impact monitoring.
Ontology bridging: Map the property-graph schema to OWL/RDF representations to enhance semantic interoperability with ontology-based MBSE tools, while retaining Neo4j for operational analytics and performance.

These developments would extend the framework from a deterministic agentic pipeline into a continuously adaptive semantic ecosystem, capable of responding autonomously to evolving system models.

7.5 Threats to Validity and Mitigation

The principal limitations of this evaluation include the dataset size and its domain specificity (aerospace CMF corpus), which may affect generalization to other engineering domains. The groundedness rubric involves human judgment, introducing subjectivity, while deterministic decoding may bias output diversity by reinforcing dominant structures.

We mitigate these threats through full reproducibility (fixed seeds, temperature = 0.0), transparent rubric disclosure in the Appendix, and publication of all evaluation scripts and error taxonomies. Future work will expand the corpus to include automotive and medical device domains and assess inter-annotator agreement on grounding scores to strengthen statistical validity.

8 CONCLUSIONS

This work presents an agentic, graph-centric architecture for Model-Based Systems Engineering that operationalizes the core ambitions of MBSE 2.0: structured requirement extraction, persistent semantics, automated impact reasoning, and governed synchronization with SysML repositories. By combining deterministic language-model extraction, a typed Neo4j knowledge graph, and autonomous agents for reasoning and editing, the proposed framework delivers a reproducible and auditable digital thread connecting natural-language requirements to executable models.

The proof-of-concept achieved strong semantic precision (0.87 weighted), production-grade link traceability (F1 = 0.95), and low hallucination ($<$ 1%) at minimal computational cost ($\sim$ \$0.016 per document). The Impact Analyzer $\rightarrow$ Graph Editor $\rightarrow$ MMS feedback loop enforces governance and auditability, while maintaining sub-second responsiveness for typical updates. Collectively, these results demonstrate that agentic reasoning over typed knowledge graphs can bridge the persistent gap between textual specifications and formal model representations, providing a practical and incremental path toward intelligent, self-evolving engineering systems aligned with the emerging SysML v2 service architecture.

Future work will extend the approach through event-driven synchronization, graph-context learning-to-link models, and ontology bridges for cross-tool semantic interoperability. These directions aim to evolve the current deterministic pipeline into a continuously adaptive ecosystem capable of maintaining model consistency autonomously while satisfying the audit requirements of safety-critical industries.

ACKNOWLEDGEMENTS

The authors thank colleagues at TCS France for early feedback on graph-schema design and synchronization policies, and the OpenMBEE community for foundational documentation and reference implementations that informed the MMS integration.

REFERENCES

AIAA SciTech (2023). Leveraging SysML v2 for integration via the new standard API and textual language. In *AIAA SciTech Forum*, pages 1–10.

Friedenthal, S. (2023). Future directions for MBSE with SysML v2. In *INCOSE International Symposium*, pages 1–12.

Hogan, A. et al. (2021). Knowledge graphs: Opportunities and challenges. *ACM Computing Surveys*, 54(4):1–38.

INCOSE (2023). SysML v2 overview and demonstration. In *INCOSE International Symposium*, pages 1–15.

Neo4j (2023–2025). *Graph Data Science Algorithms: Centrality, Community, Paths*. Neo4j, Inc. Documentation v2.5.

Norheim, T. et al. (2024). Large language models in requirements engineering: Evaluating quality, completeness, and trust. *Empirical Software Engineering*, 29(4):1–32.

Object Management Group (2023–2024). *Systems Modeling Language v2: Language and API & Services Specification*. OMG. Version 2024-05.

OpenMBEE (2020). Assisted authoring of MBSE documents. In *OpenMBEE Community Workshop*, pages 1–8.

OpenMBEE Working Group (2020–2024). OpenMBEE and MMS overview. https://www.openmbee.org/. Accessed: 2025-01-20.

Poulsen, N. et al. (2025). Artificial intelligence in systems and model-based engineering: A systematic literature review. *Journal of Systems and Software Engineering*. In press.

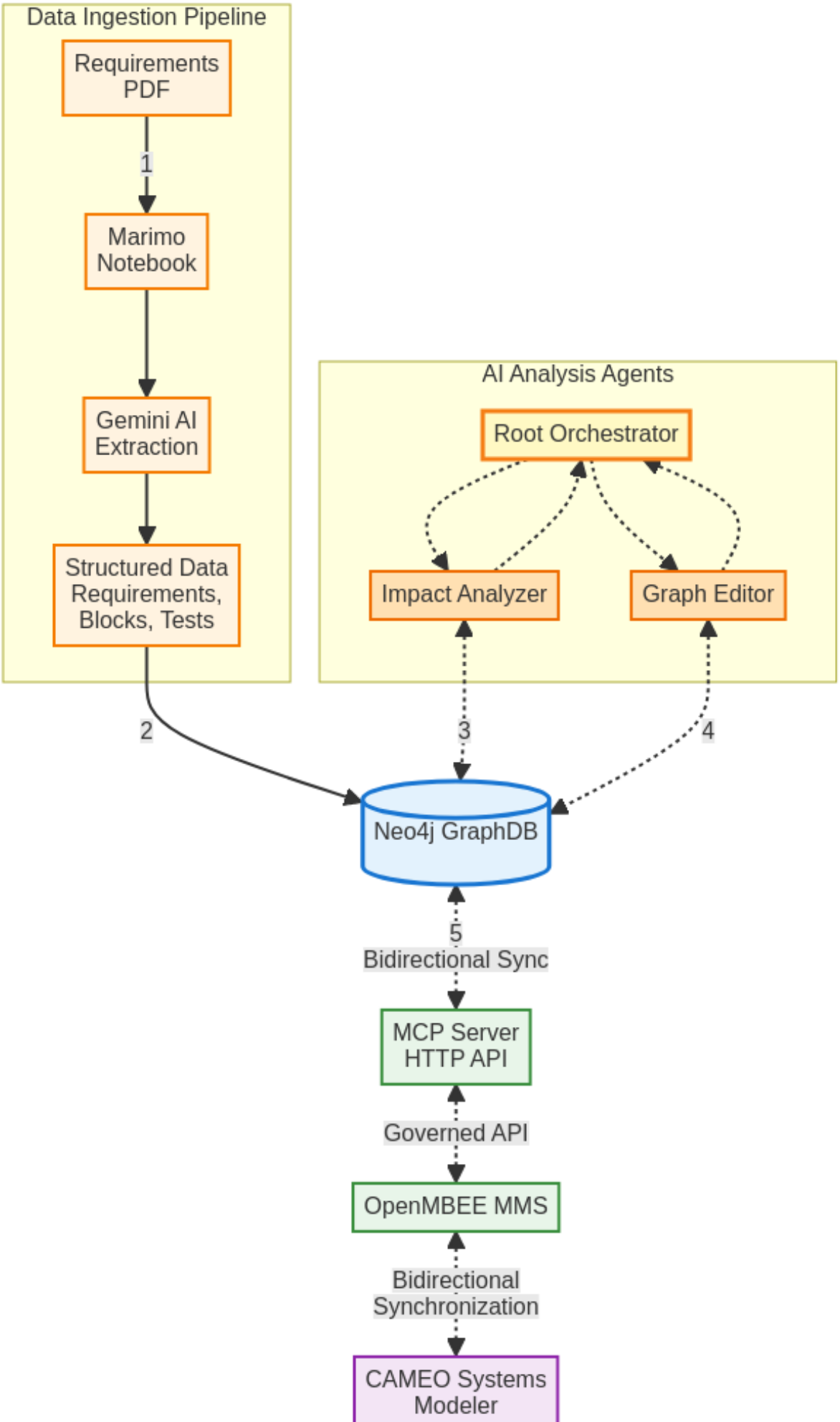
Ryś, P. et al. (2024). Ontology-driven model management for systems engineering. *Journal of Systems Engineering*, 12(3):45–67.

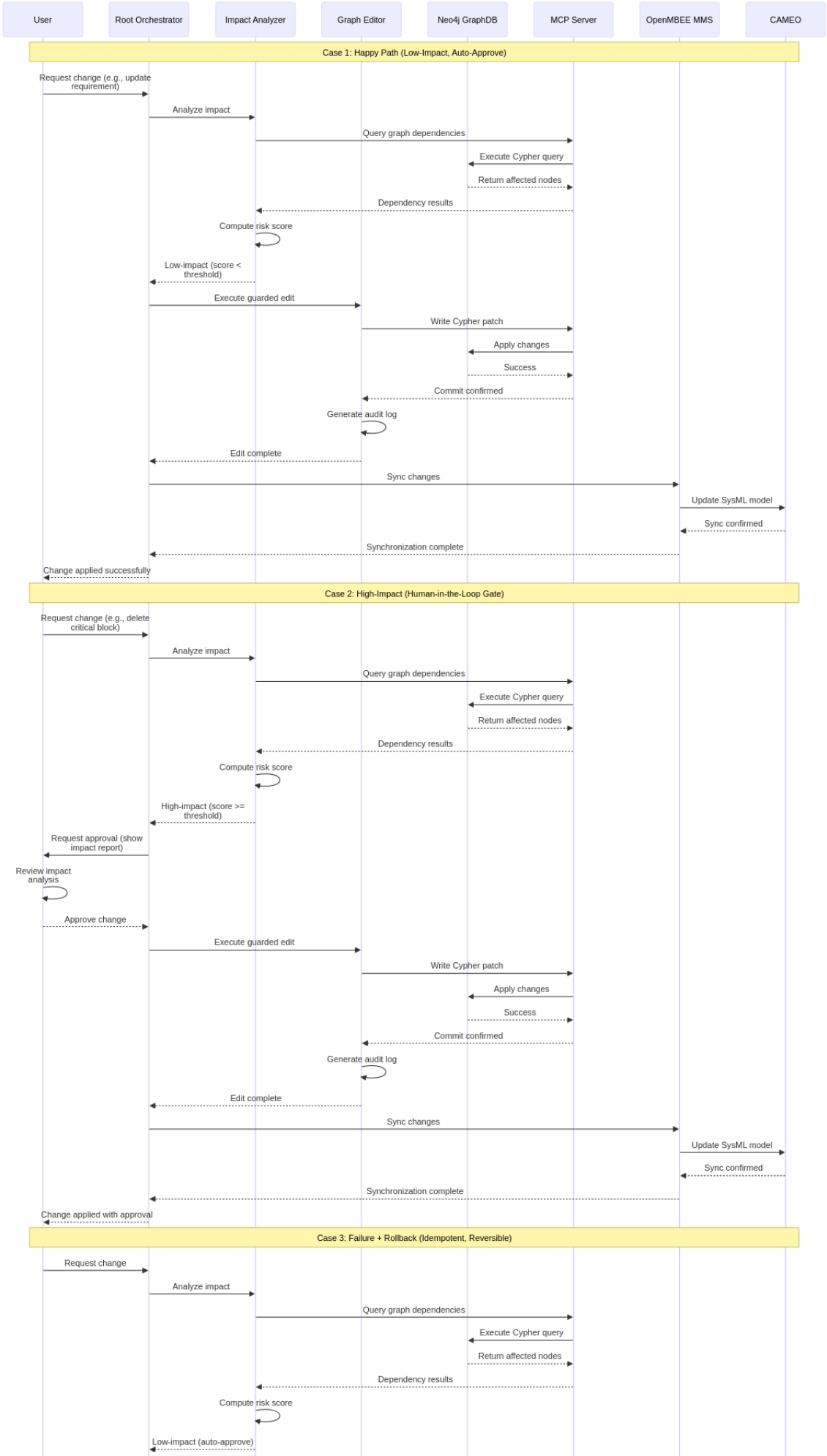
Santos, J. et al. (2024). Large language models in requirements engineering: A systematic literature review. *Requirements Engineering*. Accepted October 2024.

Schuchter, M. (2025). Application of artificial intelligence in model-based systems engineering of automated production systems. *Procedia CIRP*, 135:1–8.

U.S. DoD Digital Engineering Working Group (2024). Guidelines for migrating from SysML v1 to SysML v2. White paper, NDIA/DE.

Zhang, Y. et al. (2025). MBSE 2.0: Toward more integrated, comprehensive, and intelligent model-based systems engineering. *Systems*, 13(7):1–25.


Data Ingestion Pipeline
Requirements PDF
1
Marimo Notebook
Gemini AI Extraction
Structured Data Requirements, Blocks, Tests
AI Analysis Agents
Root Orchestrator
Impact Analyzer
Graph Editor
2
3
4
Neo4j GraphDB
5
Bidirectional Sync
MCP Server HTTP API
Governed API
OpenMBEE MMS
Bidirectional Synchronization
CAMEO Systems Modeler


User
Root Orchestrator
Impact Analyzer
Graph Editor
Neo4j GraphDB
MCP Server
OpenMBEE MMS
CAMEO
Case 1: Happy Path (Low-Impact, Auto-Approve)
Request change (e.g., update requirement)
Analyze impact
Query graph dependencies
Execute Cypher query
Return affected nodes
Dependency results
Compute risk score
Low-impact (score < threshold)
Execute guarded edit
Write Cypher patch
Apply changes
Success
Commit confirmed
Generate audit log
Edit complete
Sync changes
Update SysML model
Sync confirmed
Synchronization complete
Change applied successfully
Case 2: High-Impact (Human-in-the-Loop Gate)
Request change (e.g., delete critical block)
Analyze impact
Query graph dependencies
Execute Cypher query
Return affected nodes
Dependency results
Compute risk score
High-impact (score >= threshold)
Request approval (show impact report)
Review impact analysis
Approve change
Execute guarded edit
Write Cypher patch
Apply changes
Success
Commit confirmed
Generate audit log
Edit complete
Sync changes
Update SysML model
Sync confirmed
Synchronization complete
Change applied with approval
Case 3: Failure + Rollback (Idempotent, Reversible)
Request change
Analyze impact
Query graph dependencies
Execute Cypher query
Return affected nodes
Dependency results
Compute risk score
Low-impact (auto-approve)

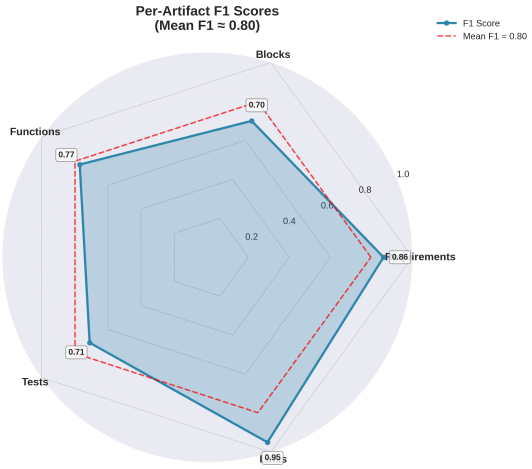


Figure 3: Radar plot of per-artifact F1 scores showing production-grade link traceability (0.95) and balanced performance across artifact types (mean F1 = 0.80).

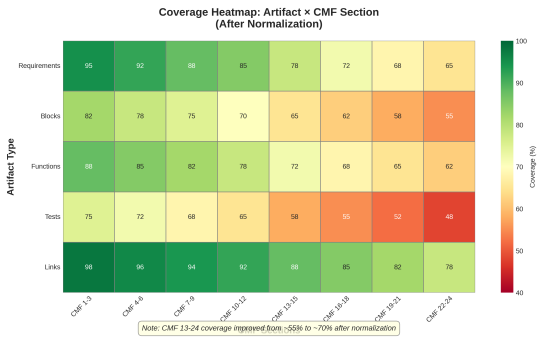


Figure 4: Coverage heatmap showing artifact recovery across CMF sections. Coverage reached approximately 70 % in late sections (CMF 13–24), indicating robust identifier normalization and link density.